

SGOF - Modelo de Estados

Documento base para ordenar la fase inicial, validar el flujo operativo y evitar inconsistencias antes de una prueba real controlada.

Fecha: 02/05/2026 | **Uso:** Biblioteca SGOF / Guia de desarrollo

Idea central: SGOF debe comportarse como el sistema operativo de la flota. El frontend puede cambiar, pero el backend y la base de datos deben conservar estados coherentes en todo momento.

1. Objetivo del modelo

Definir un conjunto claro de estados y transiciones para pedidos, viajes y taxis. El objetivo no es agregar complejidad, sino impedir errores operativos: doble asignacion, viajes abiertos sin taxi, taxis ocupados que reciben otro pedido o estados que no cierran entre si.

El modelo sirve como mapa de control para desarrollo, pruebas y futuras mejoras. Cada nueva funcion debe respetar este flujo.

2. Estados de pedido

Estado	Significado operativo	Uso principal
pendiente	Pedido creado y esperando taxi.	Entrada inicial del flujo.
asignado	Pedido con taxi asignado, esperando aceptacion o avance.	Despacho en curso.
en_proceso	El pedido ya fue tomado y esta asociado a un viaje activo.	Operacion activa.
finalizado	El pedido fue resuelto y su viaje termino.	Cierre correcto.
cancelado	El pedido fue cancelado por operador, cliente o regla operativa.	Cierre no exitoso.
sin_unidad	No hay taxi disponible o no se pudo asignar unidad.	Control de demanda no atendida.

3. Estados de viaje

Estado	Significado operativo	Uso principal
pendiente	Viaje creado pero todavia no ejecutado.	Estado inicial interno.
en_camino_origen	Taxi acepto y se dirige al origen.	Despacho aceptado.
en_origen	Taxi llego al punto de origen.	Espera / contacto con cliente.
en_viaje	Cliente a bordo y viaje en ejecucion.	Operacion principal.
finalizado	Viaje completado.	Cierre exitoso.
cancelado	Viaje cancelado antes de terminar.	Cierre no exitoso.
no_show	Taxi llego, pero el cliente no aparecio.	Cierre operativo especial.

4. Estados de taxi

Estado	Significado operativo	Uso principal
disponible	Taxi libre para recibir asignaciones.	Puede entrar en cola o proximidad.
asignado	Taxi reservado temporalmente para un pedido.	Esperando aceptacion, rechazo o timeout.
ocupado	Taxi en traslado aceptado o viaje activo.	No debe recibir otro viaje.
inactivo	Taxi fuera de servicio.	No participa en despacho.

5. Flujo operativo principal

Este es el flujo minimo que debe funcionar de punta a punta para una prueba real controlada:

```
PEDIDO.pendiente -> asignacion PEDIDO.asignado + TAXI.asignado -> taxi acepta  
PEDIDO.en_proceso + VIAJE.en_camino_origen + TAXI.ocupado -> llega al origen  
VIAJE.en_origen -> inicia viaje VIAJE.en_viaje -> finaliza PEDIDO.finalizado +  
VIAJE.finalizado + TAXI.disponible
```

6. Flujos alternativos obligatorios

Rechazo o timeout:

```
PEDIDO.asignado + TAXI.asignado -> rechazo / timeout TAXI.disponible ->  
reasignacion a otro taxi o vuelve a pendiente controlado
```

Sin unidad:

```
PEDIDO.pendiente -> no hay taxis disponibles PEDIDO.sin_unidad
```

Cancelacion:

```
Cualquier estado activo -> cancelar PEDIDO.cancelado + VIAJE.cancelado si existe +  
TAXI.disponible si estaba reservado/ocupado
```

7. Reglas de negocio que no se pueden romper

#	Regla
1	Un taxi no puede tener dos viajes activos al mismo tiempo.
2	Un pedido no puede tener dos taxis activos al mismo tiempo.
3	Un viaje finalizado debe cerrar tambien el pedido y liberar el taxi.
4	El frontend no debe ser la fuente de verdad. La verdad esta en backend y base de datos.
5	Si se recarga la pantalla o se reinicia el backend, el estado debe recuperarse desde la base.
6	Si no hay GPS, el sistema no debe bloquearse. Debe permitir despacho por cola o manual.

8. Eventos recomendados para auditoria

Cada cambio importante debe dejar rastro. Esto permite diagnosticar problemas, medir tiempos y construir reportes reales.

Evento	Cuando ocurre
pedido_creado	Cuando entra un pedido nuevo.
taxi_asignado	Cuando el sistema u operador asigna una unidad.
taxi_acepta	Cuando el taxi confirma la asignacion.
taxi_rechaza	Cuando el taxi rechaza manualmente.
timeout_asignacion	Cuando vence el tiempo sin respuesta.
reasignacion	Cuando el sistema pasa el pedido a otro taxi.
viaje_iniciado	Cuando comienza la ejecucion del viaje.
viaje_finalizado	Cuando termina correctamente.
viaje_cancelado	Cuando se cancela.
sin_unidad	Cuando no hay taxis disponibles.

9. Checklist de validacion antes de prueba real

- Crear pedido y verlo como pendiente.
- Asignar taxi automatico y manual.
- Ver taxi asignado y contador real desde fecha_hora_asignacion.
- Aceptar asignacion y pasar a viaje activo.
- Rechazar y reasignar sin duplicar taxi ni viaje.
- Probar timeout y reasignacion.
- Finalizar viaje y liberar taxi.
- Cancelar pedido/viaje y liberar taxi.
- Refrescar pantalla y confirmar que todo sigue coherente.
- Simular taxi sin GPS y confirmar que el sistema no se rompe.

10. Decision estrategica

Durante esta fase, el objetivo no es construir SubaYa ni hacer marketing masivo. El objetivo es que SGOF quede formalizado, estable y capaz de soportar una prueba real chica. SubaYa debe entrar despues como acelerador de demanda, no como sustituto del sistema operativo.

Regla final: SGOF esta listo para avanzar cuando pueda operar un flujo real sin romperse ante rechazo, timeout, cancelacion, falta de GPS o recarga de pantalla.